



**Gabriel Fortunato de Freitas  
Lais Vitória Meris  
Mariana Rosângela Patricio  
Wellington da Silva Serafim  
Professor: Daniel**

**Desenvolvimento de sistemas  
Execução de Testes**

**JOINVILLE  
2026**

## Teste Não Funcionais (Performance, Carga, Estresse)

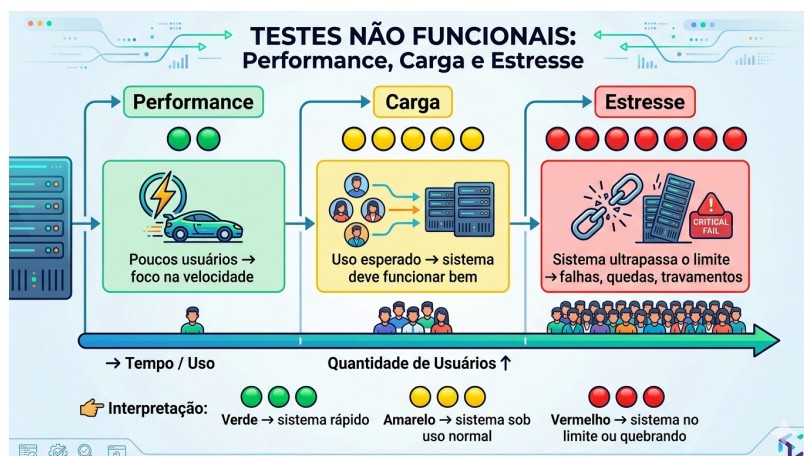
### Por quê esse tópico é importante?

Os testes não funcionais de performance, carga e estresse são essenciais para garantir que um sistema não apenas funcione corretamente, mas que também seja **rápido, estável e confiável sob diferentes condições de uso**. Em sistemas reais, especialmente aplicações web, o número de usuários pode variar drasticamente, e falhas nesse contexto podem causar lentidão, travamentos ou até a indisponibilidade completa do serviço.

### Como esses testes funcionam?

Esses testes funcionam através da simulação de múltiplos usuários e requisições simultâneas, utilizando ferramentas específicas para gerar carga no sistema. Durante os testes, são coletadas métricas como tempo de resposta (latência) e capacidade de processamento (throughput), permitindo analisar o comportamento do sistema em diferentes cenários.

O principal objetivo não é apenas observar se o sistema falha, mas identificar gargalos de desempenho (bottlenecks), ou seja, os pontos da arquitetura que limitam sua performance, como banco de dados, servidor ou rede. Dessa forma, é possível otimizar o sistema de maneira eficiente.



## Perguntas

1. Como identificar se a lentidão de um sistema é causada pelo banco de dados ou pelo servidor?
2. Um sistema pode passar em testes de carga e ainda falhar em produção? Por quê?
3. Qual a relação entre latência alta e experiência do usuário?
4. Por que testar além do limite (teste de estresse) é importante mesmo que esse cenário não seja esperado?
5. Como o aumento de usuários simultâneos pode impactar o throughput do sistema?

## Exemplos de Testes:

### Teste 1 – Performance

- **Cenário:** Usuário acessa a página inicial do sistema
- **Como Testar:** Medir o tempo de carregamento da página com poucos usuários (1 a 5)
- **Objetivo:** Garantir que o sistema responda rapidamente (ex: menos de 2 segundos)

### Teste 2 – Carga

- **Cenário:** 100 usuários acessando o sistema ao mesmo tempo.
- **Como Testar:** Simular múltiplos acessos simultâneos usando ferramenta de teste.
- **Objetivo:** Verificar se o sistema suporta o uso normal sem lentidão ou falhas.

### Teste 3 – Estresse

- **Cenário:** 1000 usuários simultâneos realizando ações no sistema.
- **Como Testar:** Aumentar gradualmente o número de usuários até o sistema apresentar falhas.
- **Objetivo:** Identificar o limite do sistema e encontrar possíveis gargalos.

O ponto mais importante desses testes não é apenas verificar se o sistema funciona sob carga, mas entender por que ele deixa de funcionar. A identificação de gargalos permite melhorias direcionadas, tornando o sistema mais eficiente, escalável e preparado para o crescimento.

## 2. Testes de Usabilidade e Acessibilidade Digital (UX)

### Testes de Usabilidade

#### Introdução

Testes de usabilidade servem para descobrir se uma pessoa consegue usar um sistema com facilidade, clareza e eficiência. O “porquê” desse teste é simples: um produto pode estar tecnicamente correto, mas ainda ser difícil de entender. Quando isso acontece, o usuário demora mais, comete erros, abandona tarefas e fica frustrado. Por isso, testar usabilidade ajuda a encontrar problemas reais antes que eles impactem usuários de verdade.

O “como” funciona por meio da observação de pessoas usando o sistema para realizar tarefas específicas, como fazer login, preencher um formulário ou comprar um produto. Durante o teste, o avaliador observa dificuldades, mede tempo, identifica erros e coleta feedback. O objetivo não é testar o usuário, mas sim o sistema. Assim, o foco está em entender onde a interface falha e como melhorá-la.



## Perguntas:

1. Um sistema pode ser funcional e ainda assim ter baixa usabilidade? Por quê?
2. Qual a diferença entre erro do usuário e erro de design?
3. Como medir se uma interface é realmente intuitiva?
4. Por que observar o comportamento do usuário é mais útil do que apenas perguntar sua opinião?
5. Um teste com poucos usuários pode revelar problemas relevantes? Explique.

## Exemplos:

| Cenário               | Como Testar                                   | Objetivo                            |
|-----------------------|-----------------------------------------------|-------------------------------------|
| Cadastro em site      | Pedir ao usuário que crie uma conta sem ajuda | Verificar clareza do fluxo e campos |
| Busca de produto      | Solicitar que encontre um item específico     | Avaliar navegação e eficiência      |
| Finalização de compra | Pedir que conclua uma compra simulada         | Identificar obstáculos no checkout  |

## Acessibilidade Digital

### Introdução

A acessibilidade digital existe para garantir que qualquer pessoa possa usar um sistema, incluindo pessoas com deficiência visual, auditiva, motora ou cognitiva. O “porquê” da acessibilidade é tornar a tecnologia inclusiva, removendo barreiras que impedem parte da população de acessar serviços, informações e experiências digitais. Um site acessível não beneficia apenas pessoas com deficiência, mas melhora a experiência de todos.

O “como” é guiado por padrões como as WCAG (Web Content Accessibility Guidelines), criadas pela W3C. Essas diretrizes orientam práticas como contraste adequado, navegação por teclado, textos alternativos em imagens e estrutura semântica clara. A acessibilidade deve ser planejada desde o início e validada com testes automáticos e humanos.



## Perguntas:

1. Acessibilidade é útil apenas para pessoas com deficiência?
2. Qual a diferença entre acessibilidade e usabilidade?
3. Um site bonito pode ser inacessível? Como?
4. Por que contraste e navegação por teclado são tão importantes?
5. É possível automatizar toda a acessibilidade? Justifique.

## Exemplos:

| Cenário                    | Como Testar                     | Objetivo                        |
|----------------------------|---------------------------------|---------------------------------|
| Navegação por teclado      | Usar Tab, Enter e Esc sem mouse | Verificar acessibilidade motora |
| Leitura com leitor de tela | Testar com NVDA ou VoiceOver    | Validar acessibilidade visual   |
| Contraste de cores         | Usar validador WCAG             | Garantir legibilidade           |



### 3. Fundamentos de Segurança da Informação nos Testes

#### Introdução

A segurança da informação nos testes de software existe porque sistemas modernos armazenam e processam dados sensíveis, como senhas, informações pessoais e dados financeiros. Se essas aplicações não forem devidamente testadas, podem apresentar vulnerabilidades que permitem ataques, vazamento de dados ou até controle indevido por terceiros.

Na prática, garantir segurança envolve testar o sistema de forma ativa, simulando ataques e validando se as proteções funcionam corretamente. Utilizam-se diferentes abordagens como SAST (análise estática), DAST (análise dinâmica) e testes manuais (pentest), garantindo que a aplicação seja segura tanto no código quanto em execução.



#### Perguntas:

1. Como a falta de validação de entrada pode comprometer toda a segurança de um sistema?
2. Qual a diferença prática entre um ataque de SQL Injection e XSS em termos de impacto?
3. Em quais situações o uso de SAST não é suficiente para garantir segurança?
4. Por que o OWASP Top 10 é considerado um padrão essencial na indústria de software?

5. Como autenticação mal implementada pode ser explorada mesmo sem vulnerabilidades aparentes no código?

**Exemplos:**

| Teste                      | Cenário                                     | Como Testar                                                    | Objetivo                                                                   |
|----------------------------|---------------------------------------------|----------------------------------------------------------------|----------------------------------------------------------------------------|
| SQL Injection              | Campo de login (usuário e senha)            | Inserir ' OR '1'='1 nos campos                                 | Verificar se o sistema bloqueia manipulação de queries e acesso indevido   |
| XSS (Cross-Site Scripting) | Campo de comentário ou formulário           | Inserir <code>&lt;script&gt;alert('XSS')&lt;/script&gt;</code> | Validar se o sistema impede execução de scripts maliciosos                 |
| Validação de Input         | Formulário de cadastro (ex: campo de email) | Inserir dados inválidos ou caracteres especiais                | Garantir que o sistema valida e sanitiza corretamente as entradas de dados |

**DESAFIO FINAL:**

**Testes Não Funcionais (Performance, Carga, Estresse)**

1. Qual é a diferença prática entre avaliar o tempo de resposta em testes de performance, o comportamento no limite esperado em testes de carga e o comportamento até a quebra do sistema em testes de estresse?

**R:** A diferença prática entre esses três tipos de teste está no nível de exigência aplicado ao sistema e no tipo de resposta que você quer analisar, por exemplo, no teste de performance você avalia quão rápido **o sistema é em condições normais**. Enquanto no teste de carga (limite esperado) Aqui você verifica se o sistema aguenta o uso real esperado. Por fim, no teste de estresse você força o sistema além do limite, até ele falhar.



### **Testes de Usabilidade e Acessibilidade Digital (UX)**

6. O que é o padrão WCAG (Web Content Accessibility Guidelines) e de que forma ele transforma a avaliação de acessibilidade em regras testáveis e binárias (passou/falhou)?

**R:** O WCAG é um padrão da W3C que define regras para tornar sites e apps acessíveis a todos, especialmente pessoas com deficiência. Ele transforma acessibilidade em regras claras e testáveis, chamadas critérios de sucesso. Cada critério pode ser verificado de forma objetiva: ou cumpre a regra, ou não. Ex: se o site tem contraste suficiente, passou; se não tem, falhou. Se pode usar só com teclado, passou; se depende do mouse, falhou.

### **Fundamentos de Segurança da Informação nos Testes**

9. Como a falta de validação de input de dados na camada de aplicação pode expor um sistema a injeções de código malicioso, como SQL Injection e Cross-Site Scripting (XSS)?

**R:** Sem validação de input, o sistema aceita dados maliciosos como comandos. Isso permite que atacantes injetem código, como SQL Injection (no banco de dados) e XSS (no navegador), comprometendo o sistema e os usuários.